TCH Event Management Module

Frontend Development Requirements

1. Project Overview

The TCH (Tech Club Hub) website requires a complete Event Management Dashboard through which the Admin and Core Team can manage events, registrations, organizers, attendance, banners, galleries and analytics.

Club Members should be able to explore events, register for events, cancel registrations and view their registration and attendance history.

The frontend must communicate with the existing Django REST Framework backend.

Base API URL

`http://127.0.0.1:8000/api/events/`

User Roles

The system has three primary roles:

1. **Admin:** Complete access to event management functionality.
2. **Core Team:** Same event management permissions as Admin. All Core Team members can manage any event, irrespective of their department or who created the event.
3. **Club Member:** Can explore published events, register for events, cancel their registrations and view their personal attendance.

---

2. Pages to Develop

PAGE 1 — Event Dashboard

Route: `/dashboard/events`

Accessible by: Admin and Core Team.

Purpose

Create a centralized dashboard displaying an overview of all event-related activities.

UI Components

1. Summary Cards:
2. Total Events
3. Upcoming Events
4. Completed Events
5. Cancelled Events
6. Total Registrations
7. Confirmed Participants
8. Present Participants
9. Absent Participants
10. Popular Events:
11. Event title
12. Category
13. Number of registrations
14. Event status
15. Category Distribution:
16. Workshop
17. Hackathon
18. Seminar
19. Webinar
20. Competition
21. Tech Talk
22. Bootcamp
23. Other
24. Registration Trends:
25. Monthly registration chart
26. Weekly registration chart
27. Quick Actions:
28. Create Event

29. View All Events
30. Manage Registrations
31. Manage Attendance

APIs

Method	Endpoint	Purpose
GET	`/analytics/`	Fetch global event analytics

API Access: Admin and Core Team.

Note: Date filters, category filters, chart interactions and weekly/monthly selection should be handled by the frontend. The current global analytics API does not necessarily support all these filters.

PAGE 2 — Event Listing / Event Management

Route: `/dashboard/events/manage`

Accessible by: Admin and Core Team.

Purpose

Display all events and allow authorized users to manage them.

UI Components

1. Event table or card layout.
2. Search by event title.
3. Filter by:
4. Category
5. Status
6. Mode
7. Event information:
8. Banner
9. Title
10. Category
11. Start date
12. End date
13. Mode

14. Venue
15. Registration count
16. Status
17. Action buttons:
18. View
19. Edit
20. Delete
21. Manage Participants
22. Manage Attendance
23. Manage Gallery
24. Create Event button.

APIs

Method	Endpoint	Purpose
GET	`/`	Fetch all accessible events
POST	`/`	Create an event
GET	`/{event_id}/`	Fetch event details
PUT	`/{event_id}/`	Completely update an event
PATCH	`/{event_id}/`	Partially update an event
DELETE	`/{event_id}/`	Delete an event

API Access:

- Admin: All operations.
- Core Team: All operations.
- Club Member: Only published events can be viewed through the event listing API.

PAGE 3 — Create / Edit Event

Route:

- `/dashboard/events/create`
- `/dashboard/events/:eventId/edit`

Accessible by: Admin and Core Team.

Purpose

Provide a complete form for creating and updating events.

Form Sections

A. Basic Information

- Event Title
- Short Description
- Full Description
- Event Category
- Event Banner

B. Event Details

- Event Mode (Online / Offline / Hybrid)
- Venue
- Meeting Link
- Start Date
- End Date

C. Registration Settings

- Registration Required (Yes/No)
- Registration Start Date
- Registration End Date
- Maximum Participants
- Registration Fee

D. Additional Information

- Eligibility
- Rules
- Prerequisites
- Contact Person
- Contact Email

E. Publishing Settings

- Featured Event
- Draft / Published / Cancelled

UI Requirements

- Validate all required fields.
- Show date validation errors.
- Display banner preview.
- Show confirmation before publishing.

- Support both creating and editing events.
- Display API validation errors near the relevant fields.

APIs

Method	Endpoint	Purpose
POST	`/`	Create event
GET	`/{event_id}/`	Fetch existing event
PUT	`/{event_id}/`	Replace event details
PATCH	`/{event_id}/`	Update selected fields
PATCH	`/{event_id}/banner/`	Upload or replace event banner

API Access: Admin and Core Team.

PAGE 4 — Event Details / Event Overview

Route: `/dashboard/events/:eventId`

Accessible by: Admin and Core Team.

Purpose

Display complete information about an individual event and provide access to its management features.

UI Components

1. Event Banner
2. Event Title and Description
3. Event Category and Mode
4. Event Dates
5. Venue / Meeting Link
6. Registration Information
7. Eligibility and Rules
8. Contact Information
9. Event Status
10. Assigned Organizers

Quick Statistics

- Total Registrations
- Confirmed Participants
- Present Participants
- Absent Participants

- Attendance Percentage
- Available Capacity

Navigation Tabs

- Overview
- Participants
- Organizers
- Attendance
- Gallery
- Analytics

APIs

Method	Endpoint	Purpose
GET	`/{event_id}/`	Fetch event information
GET	`/{event_id}/analytics/`	Fetch event statistics
GET	`/{event_id}/organizers/`	Fetch assigned organizers

API Access: Admin and Core Team.

PAGE 5 — Registration Management / Participants

Route: `/dashboard/events/:eventId/participants`

Accessible by: Admin and Core Team.

Purpose

Manage all participants registered for a particular event.

UI Components

Create a participant management table with:

- Participant Name
- Student ID
- Email
- Registration Date
- Registration Status
- Attendance Status
- Certificate Status

Features

1. Search participants.
2. Filter by registration status.
3. View participant details.
4. Confirm registrations.
5. Reject registrations.
6. Export participant information through CSV.
7. Display registration statistics.

APIs

Method	Endpoint	Purpose
GET	`/{event_id}/participants/`	Fetch event participants
PATCH	`/registrations/{registration_id}/status/`	Update registration status
GET	`/{event_id}/analytics/export/`	Download event registration report

API Access: Admin and Core Team.

PAGE 6 — Organizer Management

Route: `/dashboard/events/:eventId/organizers`

Accessible by: Admin and Core Team.

Purpose

Allow Admin and Core Team members to assign other Core Team members as organizers of an event.

UI Components

1. Assigned Organizer List.
2. Assign Organizer button.
3. Organizer profile card/table.
4. Organizer Name.
5. Email.
6. Core Team Department.
7. Responsibility.

Features

- Search available Core Team members.
- Assign a Core Team member.
- Update organizer responsibility.

- Remove an organizer.
- Display assigned organizers.

APIs

Method	Endpoint	Purpose
GET	`/{event_id}/organizers/`	Fetch organizers
POST	`/{event_id}/organizers/`	Assign organizer
PATCH	`/organizers/{organizer_id}/`	Update organizer details
DELETE	`/organizers/{organizer_id}/`	Remove organizer

Assign Organizer Request

```
{
    "member": 5,
    "responsibility": "Technical coordination"
}
```

API Access: Admin and Core Team.

Important: Only users with the `core_team` role can be assigned as organizers.

PAGE 7 — Attendance Management

Route: `/dashboard/events/:eventId/attendance`

Accessible by: Admin and Core Team.

Purpose

Provide an interface to manage attendance for all confirmed participants.

UI Components

Attendance Summary Cards

- Total Confirmed Participants
- Present
- Absent
- Not Marked

Attendance Table

Participant	Student ID	Email	Attendance
Student A	CLUB000001	student@example.com	Present
Student B	CLUB000002	student2@example.com	Absent

Features

1. Mark individual attendance.
2. Bulk attendance marking.
3. Change Present to Absent and vice versa.
4. Search participants.
5. Filter by attendance status.
6. Display attendance percentage.

APIs

Method	Endpoint	Purpose
GET	`/{event_id}/attendance/`	Fetch event attendance
PATCH	`/attendance/{registration_id}/`	Update individual attendance
POST	`/{event_id}/attendance/bulk/`	Update multiple attendance records

Individual Attendance Request

```
{
    "attendance_status": "present"
}
```

Bulk Attendance Request

```
{
    "attendance": [
        {
            "registration_id": 1,
            "attendance_status": "present"
        },
        {
            "registration_id": 2,
            "attendance_status": "absent"
        }
    ]
}
```

API Access: Admin and Core Team.

PAGE 8 — Event Gallery Management

Route: `/dashboard/events/:eventId/gallery`

Accessible by: Admin and Core Team.

Purpose

Manage photographs and images associated with an event.

UI Components

1. Image Grid.
2. Image Preview.
3. Upload Image button.
4. Caption Editor.
5. Delete Image button.
6. Image Upload Progress.

Features

- Upload multiple images (individual requests are acceptable).
- Preview images.
- Add or edit captions.
- Delete images.
- Display previously uploaded images.

APIs

Method	Endpoint	Purpose
GET	`/{event_id}/gallery/`	Fetch gallery images
POST	`/{event_id}/gallery/`	Upload gallery image
PATCH	`/gallery/{image_id}/`	Update image/caption
DELETE	`/gallery/{image_id}/`	Delete image

Upload Format

Use `multipart/form-data`.

Fields:

- `image`
- `caption`

API Access:

- Admin: Full access.
- Core Team: Full access.
- Public visitors: Can view published event galleries.

PAGE 9 — Member Events Page

Route: `/events`

Accessible by: Public visitors and Club Members, subject to the published-event visibility rules.

Purpose

Display all published events available for members.

UI Components

1. Featured Events Section.
2. Upcoming Events.
3. Event Cards.
4. Search.
5. Category Filters.
6. Event Mode Filters.
7. Event Date.
8. Event Banner.
9. Registration Status.

Event Card

Each card should display:

- Event Banner
- Event Title
- Category
- Date
- Mode
- Venue
- Short Description
- View Details button

APIs

Method	Endpoint	Purpose
GET	`/`	Fetch accessible events

Method	Endpoint	Purpose
GET	`/{event_id}/`	Fetch event details

API Access:

- Public access depends on the configured backend permissions.
- Club Members can view published events.
- Admin/Core Team can view all events.

PAGE 10 — Event Registration Page

Route: `/events/:eventId`

Accessible by: Public visitors for event viewing; Club Members for registration.

Purpose

Display complete event information and allow eligible Club Members to register.

UI Components

1. Event Banner.
2. Event Description.
3. Event Date and Time.
4. Venue / Meeting Link.
5. Eligibility.
6. Rules.
7. Prerequisites.
8. Registration Fee.
9. Available Seats (where applicable).
10. Register Now button.

Features

- Show Register button to authenticated Club Members.
- Redirect unauthenticated users to login.
- Show registration confirmation dialog.
- Show registration success message.
- Display registration status.
- Hide or disable registration when registration is closed.
- Display Cancel Registration where applicable.

APIs

Method	Endpoint	Purpose
GET	`/{event_id}/`	Fetch event details
POST	`/{event_id}/register/`	Register for event
POST	`/{event_id}/cancel-registration/`	Cancel registration

API Access:

- GET: According to event visibility permissions.
- POST Register: Club Members.
- POST Cancel: Club Members, for their own registration only.

PAGE 11 — My Registrations

Route: `/my-registrations`

Accessible by: Club Members.

Purpose

Allow members to view and manage their own event registrations.

UI Components

Create a table or card layout displaying:

- Event Name
- Event Banner
- Event Date
- Registration Date
- Registration Status
- Attendance Status
- View Event button
- Cancel Registration button

Features

- Display all personal registrations.
- Filter upcoming and past events.
- Filter by registration status.
- Cancel an eligible registration.
- Show confirmation before cancellation.

APIs

Method	Endpoint	Purpose
GET	`/my-registrations/`	Fetch personal registrations
POST	`/{event_id}/cancel-registration/`	Cancel own registration

API Access: Club Members only.

PAGE 12 — My Attendance

Route: `/my-attendance`

Accessible by: Club Members.

Purpose

Allow members to see their attendance history across all events.

UI Components

1. Total Events Attended.
2. Present Count.
3. Absent Count.
4. Event Attendance History.

Attendance Table

- Event Name
- Event Date
- Attendance Status
- Attendance Date/Time

APIs

Method	Endpoint	Purpose
GET	`/my-attendance/`	Fetch personal attendance history

API Access: Club Members.

PAGE 13 — Analytics & Reports

Route: `/dashboard/events/analytics`

Accessible by: Admin and Core Team.

Purpose

Provide detailed event performance reports.

UI Components

1. Overall Event Statistics.
2. Registration Statistics.
3. Attendance Statistics.
4. Popular Events Chart.
5. Category Distribution Chart.
6. Registration Trends.
7. Event-wise Analytics.
8. CSV Export buttons.

Charts

Use Recharts or another React charting library.

Recommended charts:

- Bar Chart: Registrations per event.
- Pie Chart: Event categories.
- Line Chart: Registration trends.
- Donut Chart: Attendance distribution.

APIs

Method	Endpoint	Purpose
GET	`/analytics/`	Fetch global analytics
GET	`/{event_id}/analytics/`	Fetch individual event analytics
GET	`/analytics/export/`	Export global registration report
GET	`/{event_id}/analytics/export/`	Export event-wise report

API Access: Admin and Core Team.

CSV Export

The frontend should provide:

- Export All Events button.
- Export Individual Event button.

The backend returns CSV files. The frontend should trigger browser downloads rather than trying to display CSV content as JSON.

---

3. Automatic Email Notifications

These notifications are handled automatically by the Django backend. No separate frontend pages or direct email APIs are required.

Trigger	Recipient	Notification
Event published	Verified active Club Members	New event announcement
Registration successful	Registered Member	Registration confirmation
Registration cancelled	Registered Member	Cancellation confirmation
Published event updated	Confirmed Participants	Event update
Event cancelled	Confirmed Participants	Cancellation announcement
Organizer assigned	Assigned Core Team Member	Organizer assignment
24 hours before event	Confirmed Participants	Event reminder

The frontend only needs to display appropriate success/error messages after user actions.

---

4. Role-Based Page Access Summary

Page	Admin	Core Team	Club Member	Public
Event Dashboard	Yes	Yes	No	No
Event Management	Yes	Yes	No	No
Create/Edit Event	Yes	Yes	No	No
Event Overview (Management)	Yes	Yes	No	No
Participants	Yes	Yes	No	No
Organizer Management	Yes	Yes	No	No
Attendance Management	Yes	Yes	No	No
Gallery Management	Yes	Yes	No	No
Member Events	Yes	Yes	Yes	Yes*
Event Registration	No**	No**	Yes	No

Page	Admin	Core Team	Club Member	Public
My Registrations	No	No	Yes	No
My Attendance	No	No	Yes	No
Analytics & Reports	Yes	Yes	No	No

*Public event visibility depends on the backend's configured permissions.

**The registration API is restricted to Club Members by the current permission rules.

---

5. Frontend Technical Requirements

Suggested Stack

- React.js
- React Router DOM
- Axios
- Tailwind CSS
- Recharts
- Lucide React
- React Hook Form (optional)

API Integration

Create a separate API service structure:

```
src/
├── services/
│   ├── eventService.js
│   ├── registrationService.js
│   ├── organizerService.js
│   ├── attendanceService.js
│   ├── galleryService.js
│   └── analyticsService.js
│
├── pages/
│   └── events/
│       ├── EventDashboard.jsx
│       ├── EventList.jsx
│       ├── EventForm.jsx
│       ├── EventDetails.jsx
│       ├── Participants.jsx
│       ├── OrganizerManagement.jsx
│       ├── AttendanceManagement.jsx
```

```
│       ├── GalleryManagement.jsx
│       ├── EventAnalytics.jsx
│       ├── PublicEvents.jsx
│       ├── EventRegistration.jsx
│       ├── MyRegistrations.jsx
│       └── MyAttendance.jsx
```

Important Development Rules

1. Use the existing authentication system and access token.
2. Do not create a new authentication mechanism.
3. Use role-based protected routes.
4. Hide management actions from unauthorized users.
5. Handle 401, 403, 404 and 500 responses appropriately.
6. Display loading indicators during API calls.
7. Add pagination wherever the backend provides paginated responses.
8. Use multipart/form-data for image uploads.
9. Do not hardcode event IDs or user IDs.
10. Refresh relevant data after create, update or delete operations.
11. Confirm before destructive actions.
12. Make all pages responsive for desktop, tablet and mobile.
13. Do not calculate authoritative registration or attendance statistics independently when the backend already provides them.

---

6. Important API Reference

All endpoints are relative to:

`/api/events/`

Module	Methods and Endpoints
Event CRUD	GET `/`, POST `/`, GET `/{id}/`, PUT `/ {id}/`, PATCH `/{id}/`, DELETE `/{id}/`
Registration	GET `/my-registrations/`, POST `/{id}/register/`, POST `/{id}/cancel-registration/`, GET `/{id}/participants/`, PATCH `/registrations/{registration_id}/status/`
Organizers	GET `/{id}/organizers/`, POST `/{id}/organizers/`, PATCH `/organizers/{organizer_id}/`, DELETE `/organizers/{organizer_id}/`
Attendance	GET `/my-attendance/`, GET `/{id}/attendance/`, PATCH `/attendance/{registration_id}/`, POST `/{id}/attendance/bulk/`
Gallery	GET `/{id}/gallery/`, POST `/{id}/gallery/`, PATCH `/gallery/{image_id}/`, DELETE `/gallery/{image_id}/`, PATCH `/{id}/banner/`

Module	Methods and Endpoints
Analytics	GET `/analytics/`, GET `/{id}/analytics/`
Export	GET `/analytics/export/`, GET `/{id}/analytics/export/`

---

7. Development Priority

The frontend should be developed in the following order:

Phase A — Admin/Core Team 1. Event Dashboard 2. Event Listing 3. Create/Edit Event 4. Event Details 5. Participants Management 6. Organizer Management 7. Attendance Management 8. Gallery Management 9. Analytics & Reports

Phase B — Club Member 1. Public Events Listing 2. Event Details and Registration 3. My Registrations 4. My Attendance

Phase C — Integration 1. Role-based navigation. 2. API integration. 3. Error handling. 4. Responsive design. 5. End-to-end testing.

Final Expected Result

The TCH website should provide a complete Event Management System where Admin and Core Team members can manage every aspect of events, while Club Members can discover events, register, and track their participation.